

Lecture 15: Memory Management and Execution Context

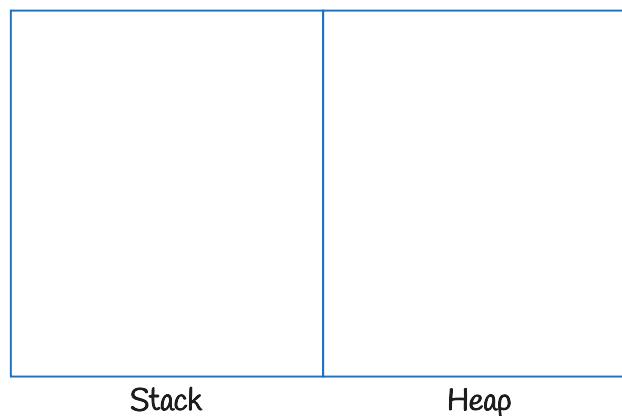
ECMAScript and Determinism:

- JavaScript operates on a strict set of rules defined by ECMAScript to prevent conflicts and ensure standard behaviour across environments.
- These rules ensure the language is deterministic, meaning the exact same input will consistently produce the exact same output.
- Different web browsers implement these standard rules using their own underlying engines (e.g., V8, SpiderMonkey, or JavaScriptCore).

Memory Architecture: Stack vs. Heap:

Whatever code runs has to be loaded into RAM first. Out of a system's total RAM, some portion is reserved for the OS and system configuration, and the rest is available for running programs.

That available memory is split into two regions: **Stack** and **Heap**.



- **Stack:** Allocates memory in a contiguous (sequential) manner.
- **Heap:** Allocates memory randomly, wherever space is available.

The Storage Misconception:

- **The Myth:** It is a common misconception that "Primitives are always stored in the Stack, and Non-Primitives are always stored in the Heap".
- **The Reality:** While that strict rule applies to languages like C++, it is not entirely accurate for JavaScript. In JavaScript, primitives can also be stored in the Heap. The split between Stack and Heap in JS is an internal engine optimization decision, not a hard language rule.

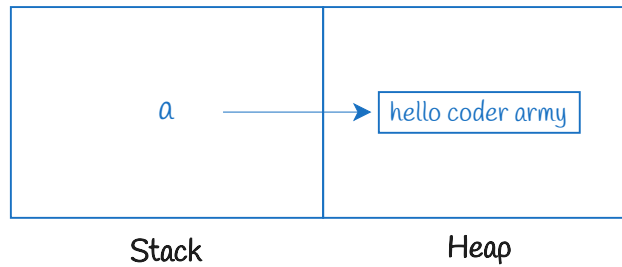
Example:

```
let a = 10;  
a = "hello coder army";
```

What happens behind the scenes?

- **The Problem:** The new string requires more memory and cannot fit into the original slot holding the number 10.
- **The Heap Allocation:** Instead of trying to overwrite it, JavaScript allocates fresh space in the Heap specifically for the new string.

- **The Stack Update:** The variable `a` in the Stack is updated to hold the memory address of this new Heap location, rather than the raw string itself.
- **Cleanup:** The old memory slot is abandoned and later cleared by the Garbage Collector.



Memory Addresses & Byte Addressability:

Whenever dynamic data is stored, memory is allocated in the Heap, and its address (reference) is stored in the Stack.

Fixed Address Size: The size of a memory address is fixed and depends entirely on the system's operating architecture.

- 32-bit OS → can address 2^{32} unique memory locations.
- 64-bit OS → can address 2^{64} unique memory locations.

Byte Addressability: Memory is allocated at the byte level, meaning every single byte in the RAM has its own unique address.

Starting Address: When larger data is stored, the engine only needs to store the starting address in the Stack. Once the engine reaches that starting byte, it can easily read the rest of the allocated block.

Garbage Collection and Optimizations:

Garbage Collector:

- This is an automated background script that identifies and deletes unreferenced data in the Heap to free up memory and prevent the RAM from crashing.
- Because the Garbage Collector consumes computational power to scan the Heap and maintain a map of free space, it can occasionally cause JavaScript to feel slow during execution.

Value Caching (Optimization):

Values like `true`, `false`, `undefined`, and `null` get fixed memory locations in the Heap, rather than a new copy being created every time they're used. One reference can be pointed to repeatedly, instead of creating separate copies.

SMI (Small Integer) Optimization:

For small numbers created repeatedly, like a loop counter in `for (let i = 0; i < 100; i++)`, allocating fresh heap memory every time would be too heavy, so small numbers are optimized to avoid this overhead.

How does the engine know whether it is an SMI or a Heap pointer?

Every stored value has a tag bit at the end so the engine instantly knows how to read it.

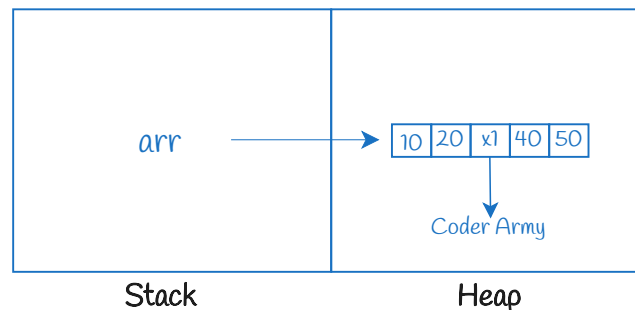
- If the last bit is 0, the value is an SMI (Small Integer). The number itself is stored directly, no Heap access needed. This is fast.
- If the last bit is 1, the value is a pointer. It's an address, and the engine follows it into the Heap to find the real value.

Array Storage in Memory:

Arrays are stored in the Heap. The Stack only holds a reference (address) to the array.

0	1	2	3	4
10	20	30	40	50

If an array holds a value that takes up more memory than a plain number (like a long string), the array may store a reference/pointer to that value instead of the value itself, keeping the array's own storage uniform.



When pushing a new element and there isn't room to expand the array in place, the engine copies the entire array to a new memory location that can fit the new size, and the variable is updated to point to that new address.

How does this work if arr is const?

- `const arr` always points to one fixed, stable reference/handle in memory.
- That fixed handle is what internally points to the actual raw array data in the Heap.

Object Storage in Memory:

An Object internally has three parts:

- **Map Pointer:** Points to the object's internal "shape" (which properties exist, and in what order).
- **Property Pointer:** Each key is paired with a numbered slot that points to where the actual value sits.
- **Element:** The actual data values that the property pointers ultimately point to.

```
{  
  "name": "Rohit",  
  "age": 20,  
  "city": "kotdwar"  
}
```

Why this layered structure (Map + Property + Element) instead of storing values directly?

- It lets the engine quickly look up where a property's value is, without scanning the whole object.
- It allows objects with the same "shape" (same property names) to share the same Map internally, an optimization, so the engine doesn't redo this layout work for every single object of the same structure.

Execution Context:

Whenever JavaScript code runs, it runs inside an Execution Context, which operates in two phases:

1. **Memory Allocation Phase:** JavaScript scans the whole code first and reserves memory.
2. **Execution Phase:** The code runs line-by-line, performs calculations, and assigns real values.

```

var a = 10;
var b = 20;

var result = addNumber(a, b);

function addNumber(num1, num2) {
  var sum = num1 + num2;
  return sum;
}

console.log(result);

```

Memory Allocation Phase	Execution Phase
a = undefined b = undefined addNumber = function result = undefined	a = 10 b = 20 result = addNumber(a, b) → 30

Hoisting: var vs let/const

let and const also go through the memory allocation phase, but they aren't initialized to undefined. Instead, they're placed in the Temporal Dead Zone (TDZ).

```

let a = 10;
const b = 20;
const addNumber = function (num1, num2) {
  return num1 + num2;
};
const result = addNumber(a, b);

```

Memory Allocation Phase	Execution Phase
a = <uninitialized> (Temporal Dead Zone) b = <uninitialized> (Temporal Dead Zone) addNumber = <uninitialized> (Temporal Dead Zone) result = <uninitialized> (Temporal Dead Zone)	a = 10 b = 20 addNumber = function result = 30

Note: A variable in the Temporal Dead Zone cannot be accessed at all, not even as undefined, until the line that actually initializes it has run. Accessing it earlier throws a ReferenceError.